

Rows, Locks & Query Plans

Everything on the Monday and Tuesday syllabus, written in plain language with worked examples, comparison tables, the traps interviewers actually set — and 46 questions with model answers.

MID-LEVEL DEPTH

GENERIC SQL

PART I · 5 TOPICS

PART II · 4 TOPICS

46 QUESTIONS

PART I · MONDAY

Structure and correctness

How data is arranged on disk so it stays consistent (normalization), how it is found quickly (indexing), and how concurrent users are stopped from corrupting each other's work (transactions and isolation).

0 Foundations & keys

Five minutes of vocabulary that the rest of the notes depends on. Interviewers use these words precisely, so you should too.

A **relation** is a table. A **tuple** is a row. An **attribute** is a column. The **degree** is the number of columns; the **cardinality** is the number of rows. A **schema** is the shape (column names and types); an **instance** is the data in it right now.

The key family

KEY	MEANING	EXAMPLE
Super key	Any set of columns that uniquely identifies a row. May contain useless extra columns.	(emp_id), (emp_id, name)
Candidate key	A minimal super key — remove any column and uniqueness breaks.	(emp_id), (email)
Primary key	The one candidate key you chose. Never NULL, never changes.	emp_id

KEY	MEANING	EXAMPLE
Alternate key	The candidate keys you didn't choose. Usually get a UNIQUE constraint.	email
Composite key	A key made of two or more columns.	(order_id, line_no)
Foreign key	A column that points at another table's primary key. Can be NULL. Enforces referential integrity.	orders.customer_id
Surrogate key	A meaningless generated id (auto-increment, UUID).	id BIGSERIAL
Natural key	A key made of real-world data.	passport_no

SAY THIS IN THE INTERVIEW

"Prefer a surrogate primary key for stability — natural keys change (people change email, companies change tax ids) and a changing PK means cascading updates everywhere. Keep the natural key as a UNIQUE constraint so the database still enforces the business rule."

Integrity constraints

- **Entity integrity** — the primary key cannot be NULL or duplicated.
- **Referential integrity** — a foreign key value must exist in the parent table (or be NULL). On delete you pick a behaviour: `RESTRICT`, `CASCADE`, `SET NULL`, `SET DEFAULT`.
- **Domain integrity** — the value fits the column's type and `CHECK` rules.
- **NULL is not a value**, it is "unknown". `NULL = NULL` is not true, it is unknown. Use `IS NULL`. This single fact causes half the bugs in this document.

1 Normalization

Normalization is the process of splitting tables so that every fact is stored exactly once. The goal is not elegance — it is that no single write can leave the database self-contradicting.

The problem it solves: three anomalies

Start with one badly designed table:

```
-- ENROLLMENT (bad)
student_id | student_name | course_id | course_name | instructor | instructor_office
-----+-----+-----+-----+-----+-----
S1         | Ankit        | C1        | Databases   | Rao        | B-204
S1         | Ankit        | C2        | Networks    | Iyer       | B-118
S2         | Meera        | C1        | Databases   | Rao        | B-204
```

- **Update anomaly** — instructor Rao moves office. His office is stored on every enrollment row, so you must update many rows. Miss one and the database now claims Rao is in two offices at once.
- **Insert anomaly** — you cannot record a new course that has no students yet, because `student_id` is part of the key and can't be NULL.
- **Delete anomaly** — if S2 drops C1 and S1 was the only other student, deleting the last row deletes the only record that course C1 exists.

All three come from the same root cause: **one table is storing facts about more than one kind of thing** (students, courses, instructors).

Functional dependency — the tool

Written `X → Y`: "if you know X, you know exactly one Y." Read it as *X determines Y*.

- `student_id → student_name` — one id, one name.
- `course_id → course_name, instructor`
- `instructor → instructor_office`

Two dependencies matter for the normal forms:

- **Partial dependency** — a non-key column depends on *part* of a composite key. Here the key is `(student_id, course_id)` but `course_name` depends on `course_id` alone.
- **Transitive dependency** — a non-key column depends on another non-key column. Here `course_id → instructor → instructor_office`.

The normal forms, in order

1NF — ATOMIC VALUES

Every cell holds a single value; no repeating groups; no arrays stuffed into a string.

```
Violates 1NF: student_id | phones
              S1         | "9990001111, 9990002222"
```

```
1NF:         a separate STUDENT_PHONE(student_id, phone) table
              with one row per phone
```

Practical note: modern engines have real array and JSON types, so "1NF" is looser than the textbook. The rule that still bites is that you cannot index or join efficiently into a comma-separated string.

2NF – NO PARTIAL DEPENDENCIES

Must be in 1NF *and* every non-key column depends on the **whole** key. Only relevant when the primary key is composite.

```
ENROLLMENT(student_id, course_id)      -- the relationship only
STUDENT(student_id, student_name)
COURSE(course_id, course_name, instructor, instructor_office)
```

The insert anomaly is now gone: a course can exist with zero students.

3NF – NO TRANSITIVE DEPENDENCIES

Must be in 2NF *and* no non-key column depends on another non-key column.

`instructor_office` depends on `instructor`, not on `course_id`, so it moves out.

```
COURSE(course_id, course_name, instructor_id)
INSTRUCTOR(instructor_id, instructor_name, office)
```

The classic one-line definition: *every non-key attribute depends on the key, the whole key, and nothing but the key*. That sentence covers 1NF, 2NF and 3NF in order.

BCNF – THE STRICTER 3NF

For every dependency `X → Y`, **X must be a super key**. 3NF has a loophole: it allows `X → Y` when Y is part of some candidate key, even if X isn't a super key. BCNF closes it.

```

CLASS(student_id, subject, teacher)
  rules: each teacher teaches exactly one subject
         a student may take a subject from any one teacher

candidate keys: (student_id, subject) and (student_id, teacher)
dependency:      teacher → subject      teacher is NOT a super key

In 3NF? Yes (subject is part of a candidate key)
In BCNF? No

Fix:  TEACHES(teacher, subject)
      TAKES(student_id, teacher)

```

TRAP

"Is every 3NF table in BCNF?" **No**. The counter-example above is the standard answer — it needs overlapping candidate keys. Also worth knowing: decomposing to BCNF is always lossless but may **not preserve all dependencies**; 3NF decomposition can always be both lossless and dependency-preserving. That is the honest trade-off, and it's why 3NF is the practical stopping point.

4NF AND 5NF – KNOW THEY EXIST

- **4NF** removes *multi-valued dependencies*: one table holding two independent one-to-many facts about the same entity. A person's (skill) and (language) in one table produces a meaningless cross-product of rows. Split into two tables.
- **5NF** (project-join normal form) handles the rare case where a table must be split three ways to remove redundancy without losing information.

Summary table

FORM	RULE IN ONE LINE	REMOVES
1NF	Atomic values, no repeating groups	Lists stuffed into cells
2NF	No partial dependency on a composite key	Duplication tied to part of the key
3NF	No transitive dependency between non-key columns	Facts about a second entity
BCNF	Left side of every dependency is a super key	The 3NF loophole with overlapping keys
4NF	No multi-valued dependency	Accidental cross-products
5NF	No join dependency not implied by keys	Rare three-way redundancy

Denormalization — deliberately going backwards

Normalization optimises for *write correctness*. It costs *read joins*. Denormalization trades some redundancy for read speed, and it is a legitimate engineering decision when you can name the price you are paying.

- **Duplicated column** — store `order.customer_name` so the order list needs no join. Also correct for *historical snapshots*: an invoice should keep the address it was shipped to, not follow the customer's current address.
- **Pre-aggregated counters** — `post.comment_count` instead of `COUNT(*)` on every page view.
- **Materialized views / summary tables** — the same idea, maintained by the database or a job.
- **Star schema** in analytics — a wide fact table plus small dimension tables, deliberately not in 3NF, because analytical queries scan and aggregate rather than update.

THE ANSWER THEY WANT

"Normalize until it hurts, denormalize until it works." Normalize by default; denormalize only with a measured read problem, and then be explicit about how the duplicate stays in sync — trigger, application write path, or a scheduled rebuild. Unowned duplicated data is how databases rot.

2 Indexing

An index is a second data structure that lets the engine find rows without reading every row. It is a copy of some columns, kept sorted or hashed, with a pointer back to the full row.

Why scanning is the enemy

Data lives in fixed-size **pages** (commonly 8 KB or 16 KB), each holding many rows. A **full table scan** reads every page. On a 10-million-row table that is tens of thousands of page reads for one row. An index turns that into a handful.

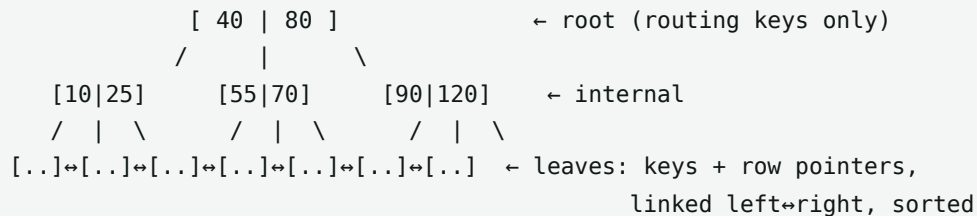
B-tree and B+ tree

Nearly every default index is a **B+ tree**. It is a balanced tree, built for disk rather than memory:

- **High fanout**. One node = one page and holds hundreds of keys, not two like a binary tree. So the tree is wide and shallow — typically **3-4 levels for millions of**

rows, meaning 3–4 page reads to find anything.

- **Always balanced.** Every leaf is at the same depth, so lookup cost is predictable: $O(\log n)$ with a very large base.
- **Data only in the leaves.** Internal nodes hold keys purely for routing. That is the "+" in B+ tree, and it keeps fanout maximal.
- **Leaves are a linked list, in sorted order.** This is the big one: once you find the start of a range you walk sideways. That is why one structure serves `=`, `<`, `>`, `BETWEEN`, `IN`, prefix `LIKE 'abc%'`, `ORDER BY`, `MIN/MAX`, and `GROUP BY`.



Hash index

Hash the key, jump straight to a bucket. **$O(1)$ average** for exact equality — slightly faster than a B-tree for that one job.

OPERATION	B+ TREE	HASH
WHERE <code>x = 5</code>	Yes, $O(\log n)$	Yes, $O(1)$
WHERE <code>x > 5</code> , <code>BETWEEN</code>	Yes	No — hashing destroys order
ORDER BY <code>x</code> without a sort	Yes	No
LIKE <code>'abc%'</code>	Yes (prefix)	No
Leftmost prefix of a composite key	Yes	No — hash covers the whole key only
MIN() / MAX() shortcut	Yes	No

Verdict: B+ trees dominate because they are good at everything; hash indexes are a narrow optimisation for equality-only lookups (and are the standard structure for in-memory hash joins and hash aggregates, which is where you meet hashing far more often).

Clustered vs non-clustered

Clustered index

The table *is* the index — rows are physically stored in key order in the leaves. One per table. Range scans on the key are sequential and very fast. Some engines cluster on the primary key automatically; others store rows in an unordered heap.

Non-clustered (secondary) index

A separate structure: indexed columns + a pointer (row id, or the clustering key). Many per table. Finding a row needs a second step to fetch the full row — the **bookmark lookup**.

CONSEQUENCE WORTH QUOTING

Where the primary key clusters the table, a **random** primary key (UUIDv4) inserts rows into the middle of the tree, causing page splits and fragmentation. A **monotonic** key (auto-increment, UUIDv7, ULID) always appends to the right edge. It also makes every secondary index bigger, since each one carries a copy of the clustering key as its pointer.

Composite indexes and the leftmost-prefix rule

An index on `(a, b, c)` is sorted by `a`, then `b` within equal `a`, then `c`. So it can be used for a prefix of the columns, left to right:

QUERY PREDICATE	INDEX (A, B, C)	WHY
<code>a = 1</code>	Used	Leftmost column
<code>a = 1 AND b = 2</code>	Used	Prefix of two
<code>a = 1 AND b = 2 AND c = 3</code>	Used fully	Whole key
<code>a = 1 AND c = 3</code>	Partly	Seeks on a; c only filters the rows found
<code>b = 2</code>	Not seekable	Skips the leading column — the index isn't sorted by b globally
<code>a > 1 AND b = 2</code>	Partly	A range on a ends the useful prefix; b can't seek after it

COLUMN-ORDER RULE

Equality columns first, then the range column, then the columns you only need for `ORDER BY` or output. A range predicate is a wall: nothing after it in the key can be used for seeking.

Covering index / index-only scan

If the index already contains every column the query needs, the engine never touches the table. That removes the bookmark lookup entirely and is often the single biggest win available.

```
SELECT status, total FROM orders WHERE customer_id = 42;
```

index (customer_id) → seek, then N random row fetches

index (customer_id, status, total) → index-only scan, zero table reads

Many engines let you add non-key payload columns with `INCLUDE (...)`, which keeps the tree narrow while still covering the query.

Selectivity: when an index is useless

Selectivity = distinct values ÷ total rows. High selectivity (email, order id) is great. Low selectivity (`gender`, `is_active` where 95% are true) is not — if a query returns a large fraction of the table, using the index means random I/O for each row plus reading most of the table anyway. Beyond a threshold (roughly 5–20% of rows, engine-dependent) the optimizer correctly chooses a sequential scan. **A sequential scan is not automatically a bug.**

Why your index isn't being used

- **Function on the column:** `WHERE YEAR(created_at) = 2026` or `WHERE LOWER(email) = ...`. The index stores `created_at`, not `YEAR(created_at)`. Rewrite as a range (`created_at >= '2026-01-01' AND created_at < '2027-01-01'`) or build an expression index. A predicate the index can use is called **sargable**.
- **Leading wildcard:** `LIKE '%son'` has no known prefix. Use full-text or a trigram index, or store a reversed column.
- **Implicit type conversion:** comparing an indexed `VARCHAR` to a number (or mismatched collations) forces a cast on the column side and kills the index.
- **Column arithmetic:** `WHERE price * 1.18 > 1000` → write `price > 1000 / 1.18`.
- **OR across different columns:** often can't use one index. Try `UNION ALL` of two indexed queries, or rely on a bitmap-OR if the engine supports it.

- **Low selectivity**, as above — and **stale statistics**, which make the optimizer mis-estimate how many rows match.
- `!=` / `NOT IN` / `IS NOT NULL` typically match most of the table, so a scan wins.

Other index types worth naming

Unique

A B-tree that also enforces a constraint. Constraint and index are the same object in most engines.

Partial / filtered

`WHERE is_deleted = false` baked into the index. Small, cheap, perfect for soft-delete and status-queue tables.

Expression / function-based

Indexes `LOWER(email)` so the case-insensitive lookup is sargable.

Bitmap

One bitmap per distinct value; ANDs and ORs are bit operations. Excellent for low-cardinality analytics, poor for OLTP writes.

Inverted / full-text

Maps each token to the rows containing it. For document and keyword search.

Spatial (R-tree)

Bounding-box tree for "points within this rectangle" geo queries.

The cost of an index

- **Every write pays.** One `INSERT` into a table with 6 indexes is 7 structures to modify, each possibly splitting pages. Indexes make reads faster and writes slower — always state this trade-off.
- **Storage and cache.** Indexes compete with data for memory. A redundant index evicts useful pages.
- **Redundancy.** An index on `(a)` is redundant if `(a, b)` exists — the prefix rule already covers it. Drop it.
- **Fragmentation and fill factor.** Random inserts split pages and leave them half-full; a lower fill factor leaves room for future inserts at the cost of size. Periodic rebuild/reorganise restores density.
- **Unused indexes** are pure cost. Every engine exposes index usage statistics — check them before adding more.

RULE OF THUMB

Index foreign keys (they are joined and checked constantly), the columns in your hottest `WHERE` clauses, and the columns you `ORDER BY` with a `LIMIT`. Then stop and measure.

3 Transactions & ACID

A transaction is a group of statements the database treats as one indivisible unit of work: all of it happens, or none of it does.

```
BEGIN;  
  UPDATE accounts SET balance = balance - 500 WHERE id = 1;  
  UPDATE accounts SET balance = balance + 500 WHERE id = 2;  
COMMIT;    -- or ROLLBACK; and it is as if nothing happened
```

`SAVEPOINT s1` marks a point you can `ROLLBACK TO s1` without abandoning the whole transaction — useful for retrying one step inside a longer unit of work.

ACID, one property at a time

A – ATOMICITY

All-or-nothing. If the second `UPDATE` fails or the server loses power mid-way, the first is undone. **How:** before changing a page, the engine writes the old value to an **undo log**. Rollback (and crash recovery) replays that log backwards. Money never vanishes between the two statements.

C – CONSISTENCY

A transaction moves the database from one valid state to another valid state. "Valid" means all declared rules hold: primary and foreign keys, unique and check constraints, triggers, and the application invariants you have encoded. **Note the honest nuance:** the database enforces the rules you declared; it cannot know an invariant you never wrote down. Consistency is a shared responsibility between schema and application.

I – ISOLATION

Concurrent transactions should not see each other's half-finished work. The ideal is that the result equals *some* serial order of the transactions. Full isolation is expensive, so engines offer levels — the whole of §4.

Once `COMMIT` returns, the change survives a crash. **How:** the **write-ahead log** (WAL / redo log).

WRITE-AHEAD LOGGING IN THREE LINES

1. The change is written to the log *before* the data page is modified on disk.
2. `COMMIT` only returns after the log record is flushed (fsync'd) to durable storage.
3. Data pages are written lazily later, at a **checkpoint**. After a crash, recovery replays the log: **redo** committed changes that hadn't reached the data files, **undo** uncommitted ones.

Why it's fast: the log is a *sequential* append, while data pages are scattered randomly. One sequential fsync buys durability for many random changes. Group commit batches several transactions into a single flush.

Locking basics

- **Shared (S) lock** — for reading. Many readers can hold it at once.
- **Exclusive (X) lock** — for writing. Only one, and it excludes S locks too.
- **Granularity** — row, page, or table. Row locks allow the most concurrency but cost memory to track; some engines **escalate** to a table lock when one transaction holds too many row locks.
- **Two-phase locking (2PL)** — the protocol that guarantees serializability: a *growing* phase where locks are acquired only, then a *shrinking* phase where they are released only. **Strict 2PL** holds every exclusive lock until commit, which is what real engines do.

Deadlocks

```
T1: UPDATE accounts SET .. WHERE id = 1;  -- holds X on row 1
T2: UPDATE accounts SET .. WHERE id = 2;  -- holds X on row 2
T1: UPDATE accounts SET .. WHERE id = 2;  -- waits for T2
T2: UPDATE accounts SET .. WHERE id = 1;  -- waits for T1 → deadlock
```

Databases detect this with a **wait-for graph**: a cycle means deadlock, and the engine kills the cheaper transaction as the *victim*, returning a deadlock error. That error is **retryable**, and your application should retry it.

How to make deadlocks rare:

- **Always touch rows in the same order** (e.g. ascending id). This single rule eliminates most deadlocks.
- Keep transactions short; never hold a transaction open across a network call or user think-time.
- Take the strongest lock you'll need up front (`SELECT ... FOR UPDATE`) rather than upgrading a shared lock later.
- Lower isolation where correctness allows; add the right indexes so fewer rows get locked.

DEADLOCK VS LIVELOCK VS STARVATION

Deadlock: two transactions wait on each other forever — nothing moves.

Livelock: they keep reacting to each other and retrying, so there is activity but no progress. **Starvation:** one transaction is repeatedly passed over while others proceed.

Pessimistic vs optimistic concurrency

Pessimistic

Lock first, then work: `SELECT ... FOR UPDATE`. Right when contention is high and a retry is expensive (seat booking, inventory decrement). Cost: blocking, and deadlock risk.

Optimistic

No lock; carry a `version` column and write with `WHERE version = :seen`. If 0 rows update, someone else won — reload and retry. Right when conflicts are rare. Cost: wasted work on conflict.

4 Isolation levels

Isolation levels are a dial between correctness and concurrency. Each level is defined by which read anomalies it permits.

The anomalies

- **Dirty read** — you read a row another transaction has changed but not committed. It may roll back, so you read a value that never existed.
- **Non-repeatable read** — you read the same *row* twice in one transaction and get different values, because someone committed an UPDATE in between.
- **Phantom read** — you run the same *range query* twice and get a different set of rows, because someone committed an INSERT or DELETE matching your predicate.

- **Lost update** — two transactions read the same value, both compute from it, both write; the second silently overwrites the first. (Read 100, both add 10, result is 110 instead of 120.)
- **Write skew** — two transactions read an overlapping set, each checks a rule that still holds in *its* snapshot, and both write different rows. Together they break the rule. Classic case: two on-call doctors, each transaction confirms "someone else is still on call", and both go off duty. This is the anomaly snapshot isolation does *not* prevent, and it needs `SERIALIZABLE` or an explicit lock.

The four standard levels

LEVEL	DIRTY READ	NON-REPEATABLE	PHANTOM	TYPICAL COST
Read Uncommitted	Possible	Possible	Possible	Lowest — almost never worth it
Read Committed	Prevented	Possible	Possible	Low — the common default
Repeatable Read	Prevented	Prevented	Possible*	Medium
Serializable	Prevented	Prevented	Prevented	Highest — blocking or retries

* Per the SQL standard. Real engines differ: some prevent phantoms at Repeatable Read using next-key/gap locks, and some implement Repeatable Read as snapshot isolation, which stops phantoms in reads but still allows write skew. If asked, say what the standard requires *and* that implementations vary — that is the mature answer.

How isolation is implemented

Lock-based

Readers take shared locks, writers exclusive ones, held to commit under strict 2PL. Range/gap locks stop phantoms. Simple and strict; readers and writers block each other.

MVCC (multi-version)

Every write creates a *new version* of the row stamped with a transaction id. Each reader sees a consistent **snapshot** of versions committed before it began. **Readers don't block writers and writers don't block readers** — the reason MVCC won. Cost: old versions accumulate and must be cleaned up (vacuum / purge), and long-running transactions hold that cleanup back.

Under MVCC, *Read Committed* takes a fresh snapshot at the start of every statement; *Repeatable Read* takes one snapshot at the start of the transaction and keeps it. That one sentence explains the whole difference in behaviour.

Choosing a level in practice

- **Read Committed** for ordinary application traffic. Understand that a value can change between two statements in your transaction.
- **Repeatable Read** for a multi-statement report that must see one consistent moment in time.
- **Serializable** for money, inventory, booking — anywhere write skew would be a real bug. Budget for serialization failures and **write a retry loop**; that is not optional.
- Or keep a low level and lock explicitly where it matters: `SELECT ... FOR UPDATE`, or a single atomic statement like `UPDATE stock SET qty = qty - 1 WHERE id = ? AND qty > 0`, which is immune to lost update because the read and write happen inside one statement.

INTERVIEW FAVOURITE

"How do you prevent lost update?" Three valid answers, and naming all three is the strong answer: (1) do the arithmetic in the database in one statement; (2) pessimistic — `SELECT ... FOR UPDATE` before computing; (3) optimistic — a version column with a compare-and-set `WHERE version = :seen`, retried on failure.

5 Interview questions — Part I

Answer out loud, then open the card. Each one ends with what the question is really testing.

24 QUESTIONS · NORMALIZATION, INDEXING, TRANSACTIONS

01 What is normalization and why do we do it?

Organising columns into tables so each fact is stored once, based on the functional dependencies between columns. The purpose is to eliminate redundancy and with it the update, insert and delete anomalies — states where one write leaves the database contradicting itself. The cost is more tables and therefore more joins on read.

Testing: whether you name the anomalies, or just recite "removes redundancy".

02 Explain 1NF, 2NF and 3NF with one example.

1NF: atomic values, no repeating groups — split a comma-separated `phones` column into its own table. 2NF: in 1NF and no non-key column depends on part of a composite key — in `ENROLLMENT(student_id, course_id, course_name)`, `course_name` depends only on `course_id`, so it moves to a `COURSE` table. 3NF: in 2NF and no non-key column depends on another non-key column — `instructor_office` depends on `instructor`, so `INSTRUCTOR` becomes its own table.

One-liner: every non-key attribute depends on *the key, the whole key, and nothing but the key*.

Testing: can you carry one concrete example through all three steps rather than reciting definitions.

03 Difference between 3NF and BCNF. Is every 3NF relation in BCNF?

BCNF requires the determinant of every functional dependency to be a super key. 3NF allows an exception: $X \rightarrow Y$ is tolerated when Y is a prime attribute (part of some candidate key) even if X is not a super key. So no — 3NF does not imply BCNF; BCNF implies 3NF.

Counter-example: `CLASS(student_id, subject, teacher)` where each teacher teaches one subject. Candidate keys are `(student_id, subject)` and `(student_id, teacher)`; the dependency `teacher → subject` has a non-super-key determinant. It is in 3NF but not BCNF.

Trade-off: BCNF decomposition is always lossless but may lose dependency preservation; 3NF can always be both.

Testing: the counter-example, and whether you know BCNF isn't free.

04 What is a functional dependency? Partial vs transitive?

$X \rightarrow Y$ means each value of X is associated with exactly one value of Y. **Partial**: a non-key column depends on only part of a composite key (a 2NF violation). **Transitive**: a non-key column depends on another non-key column, so `key → A → B` (a 3NF violation).

Testing: the vocabulary that the normal forms are actually defined in.

05 When would you denormalize?

When a measured read path is too slow because of joins or aggregates, and the write path can keep the duplicate honest. Common forms: duplicating a display column to avoid a join; a maintained counter instead of `COUNT(*)`; a materialized view or summary table; a star schema for analytics. Also for deliberate *historical snapshots* — an invoice must keep the address as it was, not follow the customer record.

Always state the price: duplicated data can drift, so name the mechanism keeping it in sync and the reconciliation job that catches drift.

Testing: engineering judgement — do you denormalize with a reason and an owner, or by habit.

06 Candidate key vs primary key vs super key.

Super key: any column set that uniquely identifies rows. Candidate key: a *minimal* super key — drop any column and uniqueness fails. Primary key: the candidate key you designate; not null, ideally immutable. The remaining candidate keys are alternate keys and normally get UNIQUE constraints.

Testing: precision. "Minimal" is the word they listen for.

07 Surrogate key or natural key?

Surrogate by default: it is stable, narrow, and never needs a cascading update when business facts change. Keep the natural key as a UNIQUE constraint so the rule is still enforced by the database. Natural keys are reasonable when they are genuinely immutable and you gain from avoiding a join. In an engine that clusters on the primary key, also prefer a monotonically increasing surrogate (auto-increment, UUIDv7) over a random UUID to avoid page splits and index bloat.

Testing: whether you connect key choice to physical storage, not just modelling taste.

08 What is an index and what does it cost?

A separate sorted (or hashed) structure over some columns plus pointers back to rows, so the engine can find rows without scanning every page. It costs storage, memory in the buffer cache, and write time: every insert, update of an indexed column, and delete must maintain every affected index. Reads get faster, writes get slower.

Testing: that you volunteer the write cost unprompted.

09 Why B+ tree instead of a binary search tree or a hash table?

Because the unit of I/O is a page, not a value. A B+ tree node fills a page and holds hundreds of keys, so the tree stays 3-4 levels deep for millions of rows — a handful of page reads. A binary tree with the same data would be ~20+ levels, each a potential random read. It is also always balanced, so worst case equals average case.

Against a hash table: hashing gives $O(1)$ equality but destroys order, so it cannot serve ranges, `BETWEEN`, `ORDER BY`, prefix `LIKE`, `MIN/MAX`, or partial-key lookups. B+ tree leaves are sorted and linked, so one structure serves all of those.

Testing: disk-awareness (fanout, page reads) and range support — the two real reasons.

10 Clustered vs non-clustered index.

A clustered index defines the physical order of the rows — the table's leaves *are* the data, so there is at most one, and range scans on the key are sequential. A non-clustered index is a separate structure holding indexed columns plus a pointer; you can have many, and using one to fetch other columns costs a second lookup into the table (bookmark lookup). That lookup is why the optimizer sometimes prefers a full scan for a query returning many rows.

Testing: whether you know the extra fetch exists — it explains a lot of plan behaviour.

11 What is a covering index?

An index containing every column the query needs — both in the predicate and in the select list — so the engine answers entirely from the index and never reads the table. Plans call it an index-only scan. It removes all the random row fetches, which is usually the dominant cost. Where supported, `INCLUDE` adds payload columns to the leaf level without widening the key.

Testing: that "covering" means covering the whole query, not just the WHERE clause.

12 You have an index on (a, b, c). Which queries can use it?

It is sorted by a, then b, then c, so any leftmost prefix seeks: `a`; `a, b`; `a, b, c`. `a = 1 AND c = 3` seeks on `a` and then only filters on `c`. `b = 2` alone cannot seek, because rows with `b = 2` are scattered across all values of `a`.

Also: a range predicate stops the seek. In `a > 10 AND b = 5`, `b` can't be used for seeking because the `a` range spans many `b` orderings. Hence the design rule: equality columns first, range column last.

Testing: the leftmost-prefix rule and the range-stops-the-seek corollary.

13 Query has an index but the plan shows a full scan. Why?

Walk the list: (1) the predicate isn't sargable — a function, arithmetic, or an implicit cast on the indexed column; (2) leading wildcard `LIKE '%x'`; (3) low selectivity — the query returns a big fraction of rows, so a sequential scan genuinely is cheaper; (4) stale statistics making the row estimate wrong; (5) the index doesn't cover the query and the optimizer prefers one scan over many random lookups; (6) an `OR` across different columns; (7) the table is tiny and it doesn't matter.

Then say: I'd confirm with `EXPLAIN ANALYZE` and compare estimated vs actual rows before changing anything.

Testing: a systematic checklist and the maturity to accept that a scan is sometimes correct.

14 What is selectivity and why does it matter?

The fraction of rows a predicate keeps (or, for a column, distinct values $\div$ row count). Highly selective predicates match few rows and benefit hugely from an index. A predicate matching 40% of a table is better served by a sequential scan, because index access does one random I/O per row plus the index reads. The optimizer estimates selectivity from statistics and histograms — wrong statistics mean wrong plans.

Testing: the link from selectivity to plan choice to statistics.

15 Where does a hash index beat a B-tree?

Only for exact equality on the full key, where its $O(1)$ probe beats $O(\log n)$ descent and its structure can be smaller. It cannot do ranges, ordering, prefix matching, or partial keys, so it is a niche choice. Hashing matters far more *inside* query execution — hash joins and hash aggregates build in-memory hash tables at run time.

Testing: that you don't oversell $O(1)$.

16 What is a transaction? Explain ACID.

A unit of work that either fully happens or fully doesn't. **Atomicity** — all or nothing, implemented with an undo log. **Consistency** — every commit leaves declared constraints and invariants satisfied. **Isolation** — concurrent transactions don't see each other's uncommitted work; the strength is set by the isolation level. **Durability** — once commit returns, the change survives a crash, guaranteed by flushing the write-ahead log before acknowledging.

Testing: whether you can attach a mechanism to each letter, not just a definition.

17 How is durability actually achieved?

Write-ahead logging. The change is appended to the log before the corresponding data page is written, and `COMMIT` returns only after that log record is fsync'd. Data pages are flushed later at a checkpoint. On restart, recovery redoes committed changes missing from the data files and undoes uncommitted ones. It is fast because the log is a sequential append — one sequential flush covers many scattered page changes — and group commit amortises the fsync across concurrent transactions.

Testing: WAL, fsync, checkpoint, redo/undo – the four words they're waiting for.

18 List the isolation levels and what each prevents.

Read Uncommitted allows dirty reads. **Read Committed** stops dirty reads; non-repeatable reads and phantoms remain. **Repeatable Read** also stops non-repeatable reads; the standard still permits phantoms. **Serializable** prevents all three and guarantees the outcome equals some serial execution.

Add the nuance: implementations differ. Some prevent phantoms at Repeatable Read with gap locks; MVCC engines often implement it as snapshot isolation, which stops phantoms on reads but still permits write skew.

Testing: the table – and whether you know the standard and reality diverge.

19 Dirty read vs non-repeatable read vs phantom read.

Dirty: you read uncommitted data that may be rolled back. **Non-repeatable:** you re-read the *same row* and its values changed because another transaction committed an UPDATE. **Phantom:** you re-run the *same range query* and the set of matching rows changed because of a committed INSERT or DELETE. Row-level locks prevent the second; you need range/gap locks or a snapshot to prevent the third.

Testing: row-level vs set-level – the distinction most candidates blur.

20 What is write skew and which level stops it?

Two transactions read an overlapping set, each verifies a constraint that still holds in its own snapshot, then each writes a *different* row. Individually valid, together they break the invariant — two doctors both go off call because each saw the other still on call. Snapshot isolation (often labelled Repeatable Read) does not prevent it, because neither transaction wrote the same row. You need `SERIALIZABLE`, or a deliberate lock that materialises the conflict (`SELECT ... FOR UPDATE` on the rows you checked, or a lock on a summary row).

Testing: a senior-flavoured question — knowing snapshot isolation isn't serializable.

21 What is MVCC and why is it popular?

Multi-version concurrency control: an update writes a new version of the row rather than overwriting it, tagged with transaction ids, and each transaction reads the snapshot of versions visible as of its start point. The payoff is that **readers never block writers and writers never block readers**, which is enormous for read-heavy workloads. The costs are storage for old versions and a cleanup process (vacuum/purge) that a long-running transaction can hold back, causing bloat.

Testing: the benefit *and* the version-cleanup cost.

22 What causes a deadlock and how do you deal with it?

Two or more transactions each hold a lock the other needs, forming a cycle in the wait-for graph. The engine detects the cycle and aborts a victim with a retryable error. Prevention: acquire locks in a consistent order (usually ascending primary key), keep transactions short and free of external calls, take the strongest lock you'll need up front instead of upgrading, index well so fewer rows are locked, and batch in small chunks. Your application should always retry a deadlock error with backoff.

Testing: ordering as the primary fix, and that retry is the app's job.

23 Optimistic vs pessimistic locking — which and when?

Pessimistic locks the row before working (`SELECT ... FOR UPDATE`): correct choice when contention is high or a retry is expensive, at the cost of blocking and deadlock risk. Optimistic carries a version and writes with `WHERE version = :seen`, retrying if zero rows changed: better when conflicts are rare and transactions are long or span a user's think-time, since holding a database lock across a user session is unacceptable.

Testing: that the choice is driven by conflict rate and transaction duration.

24 Two users transfer money at once. How do you keep the balance correct?

Wrap both updates in one transaction so it's atomic. Then prevent lost update by one of: a single arithmetic statement (`UPDATE accounts SET balance = balance - 500 WHERE id = 1 AND balance >= 500`, checking the affected row count); `SELECT ... FOR UPDATE` on both rows before computing; or a version column with compare-and-set. Lock the two accounts in a deterministic order — ascending id — so concurrent opposite transfers cannot deadlock. Use `SERIALIZABLE` if a cross-row invariant is involved, and retry serialization and deadlock failures.

Testing: the whole of Part I in one scenario. Say "atomic statement", "consistent lock order", "retry".

Speed and throughput

How rows from different tables are combined (joins), how the engine decides on a strategy (optimization), how to read the decision it made (execution plans), and how to stop the connection layer from becoming the bottleneck (pooling).

6 Joins

A join matches rows from two tables using a condition. Two separate things are worth knowing: the *logical* join types (what result you asked for) and the *physical* join algorithms (how the engine actually does it).

Logical join types

TYPE	RETURNS	USE IT FOR
INNER JOIN	Only rows matching on both sides	The default. Orders with a known customer.
LEFT [OUTER] JOIN	All left rows; right columns NULL where no match	"All customers, with their orders if any."
RIGHT [OUTER] JOIN	Mirror of LEFT	Rare — most people rewrite it as a LEFT join.
FULL OUTER JOIN	All rows from both, NULLs where unmatched	Reconciling two systems.
CROSS JOIN	Cartesian product, every pair	Generating combinations, calendars. Accidentally: a missing join condition.
SELF JOIN	A table joined to itself with aliases	Employee → manager, hierarchies.
NATURAL JOIN	Joins on all same-named columns automatically	Avoid. Adding a column later silently changes the result.
Semi-join	Left rows that <i>have</i> a match, no duplication	WHERE EXISTS (...) / IN (...)
Anti-join	Left rows that have <i>no</i> match	WHERE NOT EXISTS (...), or LEFT JOIN + IS NULL

Four traps that come up constantly

1 – ON VS WHERE ON AN OUTER JOIN

```
SELECT c.id, o.id
FROM customers c
LEFT JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'PAID';      -- silently an INNER join!
```

The `WHERE` runs *after* the join, and unmatched customers have `o.status = NULL`, which fails the test — so they are filtered out. Put the condition in the `ON` clause to filter the right table *before* matching:

```
LEFT JOIN orders o ON o.customer_id = c.id AND o.status = 'PAID'
```

Rule: for an outer join, conditions on the *optional* side belong in `ON`; conditions on the *preserved* side belong in `WHERE`. (For an inner join it makes no difference to the result.)

2 – NOT IN WITH NULLS

```
SELECT * FROM customers
WHERE id NOT IN (SELECT customer_id FROM orders);  -- returns ZERO rows if any
                                                    customer_id is NULL
```

`x NOT IN (1, 2, NULL)` evaluates to *unknown*, never true, so nothing passes. Use `NOT EXISTS`, which handles NULL correctly and usually optimises into a proper anti-join:

```
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

3 – FAN-OUT: JOINS MULTIPLY ROWS BEFORE AGGREGATION

```
SELECT o.id, SUM(i.amount) AS items, SUM(p.amount) AS paid
FROM orders o
JOIN order_items i ON i.order_id = o.id      -- 3 items
JOIN payments    p ON p.order_id = o.id      -- 2 payments
GROUP BY o.id;  -- 6 rows: both sums are inflated 2x and 3x
```

Joining two one-to-many children of the same parent produces their cross-product. Fix by aggregating each child in its own subquery/CTE first and joining the single-row results, or by using `SUM(DISTINCT ...)` only when you're certain of the semantics.

Comma-separated tables in `FROM` with a forgotten predicate gives every pair. Always use explicit `JOIN ... ON` syntax; a missing `ON` is then a syntax error rather than a 10-billion-row result.

Physical join algorithms

The optimizer picks one per join. Knowing them is how you read a plan.

ALGORITHM	HOW IT WORKS	ROUGH COST	CHOSEN WHEN
Nested loop	For each row of the outer table, scan the inner table for matches.	$O(N \times M)$	Only sane for a tiny outer side.
Index nested loop	For each outer row, do an index lookup on the inner table.	$O(N \times \log M)$	The workhorse of OLTP. Small outer side + an index on the inner join column. Best for queries returning few rows.
Block nested loop	Buffer a block of outer rows and scan the inner table once per block.	fewer passes	Fallback when no index exists and hash join isn't available.
Hash join	Build an in-memory hash table on the smaller input, then probe it with each row of the larger.	$O(N + M)$	Large unsorted inputs with an equality condition. Cannot do <code></></code> joins. Spills to disk if the build side exceeds working memory.
Sort-merge join	Sort both inputs on the join key, then walk them together once.	$O(N \log N + M \log M)$	Both inputs already sorted (e.g. arriving from indexes), very large inputs, or non-equality joins. Output is sorted, which can pay for a later <code>ORDER BY</code> .

ONE LINE TO REMEMBER

Few rows → index nested loop. Many rows → hash join. Already sorted or non-equality → merge join. A nested loop over a huge outer side is the single most common cause of a query that runs in 40 seconds instead of 40 milliseconds — and it usually means the optimizer underestimated the row count.

Join order

The optimizer also chooses the *order* in which tables are joined, because joining the two most selective tables first keeps intermediate results small. With n tables the search space grows factorially, so optimizers use dynamic programming plus

heuristics, and above a threshold (often ~8–12 tables) they stop searching exhaustively and take a greedy plan. This is why very wide joins get unpredictable, and why breaking a monster query into staged steps sometimes wins.

7 Query optimization

SQL is declarative: you describe the result, not the method. The optimizer's job is to pick the cheapest method. Optimizing a query mostly means giving the optimizer better information and fewer obstacles.

What happens when you send a query

1. **Parse** — syntax check, build a parse tree.
2. **Bind / resolve** — check that tables and columns exist and you have permission; resolve types.
3. **Rewrite** — apply equivalence transformations: expand views, flatten subqueries, push predicates down, prune unused columns, simplify constant expressions.
4. **Optimize** — enumerate candidate physical plans (access method per table, join algorithm, join order), estimate the cost of each using statistics, keep the cheapest.
5. **Execute** — run the chosen plan tree, pulling rows through the operators.

Steps 1–4 can be skipped for a repeated query via a **plan cache**; that's part of why prepared statements are faster.

Cost-based optimization and statistics

Cost is an abstract number combining estimated I/O and CPU. It is only as good as the **statistics**: row counts, distinct values per column, null fraction, most-common values, and **histograms** describing the data distribution.

THE SINGLE MOST COMMON ROOT CAUSE

Stale statistics. After a bulk load or a large delete, the optimizer thinks a table has 1,000 rows when it has 10 million, picks a nested loop, and the query falls off a cliff. Running the engine's analyze/statistics-update command is the first thing to try on a query that "was fast yesterday". Also beware **correlated columns** — the optimizer usually assumes independence, so `city = 'Pune' AND state = 'Maharashtra'` gets multiplied into a wild underestimate unless you declare multi-column statistics.

Rewrites the optimizer does for free

- **Predicate pushdown** — filter as early as possible, before joins, so less data flows upward.
- **Projection pruning** — stop carrying columns nobody selected.
- **Constant folding** — evaluate `2 * 3` once, at plan time.
- **Subquery flattening** — turn an `IN` subquery into a semi-join.
- **Join elimination** — drop a joined table entirely if a foreign key proves the join can't change the result and no column from it is used.
- **Limit pushdown / early stop** — stop producing rows once `LIMIT` is satisfied.

Anti-patterns and their rewrites

ANTI-PATTERN	WHY IT'S SLOW	REWRITE
<code>SELECT *</code>	Carries unused columns, defeats covering indexes, breaks when the schema changes	Name the columns you need
<code>WHERE YEAR(d) = 2026</code>	Not sargable — the index has <code>d</code> , not <code>YEAR(d)</code>	<code>d >= '2026-01-01' AND d < '2027-01-01'</code>
<code>WHERE a = 1 OR b = 2</code>	Often can't use one index	<code>UNION ALL</code> of two indexed queries (or rely on bitmap OR)
<code>NOT IN (subquery)</code>	Wrong with NULLs and often slower	<code>NOT EXISTS</code>
<code>OFFSET 100000</code> <code>LIMIT 20</code>	Reads and discards 100,000 rows — gets slower every page	Keyset pagination: <code>WHERE (created_at, id) < (:last_ts, :last_id) ORDER BY created_at DESC, id DESC LIMIT 20</code>
<code>SELECT DISTINCT</code> sprinkled everywhere	Usually hiding accidental fan-out; forces a sort or hash	Fix the join; or use <code>EXISTS</code> for a semi-join
N+1 queries from the app	1 query for the list, then 1 per row — latency × N	One join, or one <code>IN (. . .)</code> batch fetch
Implicit cast (<code>id = '42'</code>)	Forces a conversion on the column side	Match the parameter type to the column
<code>COUNT(*)</code> on a huge table per request	Full scan every time	Maintained counter, approximate count, or cache it

Subqueries, CTEs and window functions

- **EXISTS vs IN**: both usually become a semi-join, so modern optimizers treat them alike. **EXISTS** short-circuits at the first match and is NULL-safe, so prefer it — especially in the negative form.
- **Correlated subquery in the SELECT list** runs once per output row. That's an N+1 hidden inside SQL. Rewrite as a join to a pre-aggregated derived table, or as a window function.
- **CTEs (WITH)** improve readability. Some engines historically *materialized* them into a temp result (an optimization fence); modern versions usually inline them unless told otherwise. Don't assume a CTE is free — check the plan.
- **Window functions** (`ROW_NUMBER() OVER (PARTITION BY ...)`) replace self-joins for "top N per group", running totals and ranking, and usually need one sort instead of one join per group.
- **WHERE vs HAVING**: **WHERE** filters rows before grouping, **HAVING** filters groups after. Filtering in **HAVING** what could have been filtered in **WHERE** means grouping rows you were going to throw away.

Structural levers, when indexes aren't enough

- **Partitioning** — split a huge table by range (usually date) or hash. The optimizer then does **partition pruning**, touching only relevant partitions, and dropping old data becomes an instant partition drop instead of a giant DELETE.
- **Materialized views / summary tables** — precompute expensive aggregates and refresh on a schedule. You trade freshness for speed.
- **Read replicas** — move reporting traffic off the primary. Accept replication lag; never read your own write from a replica without care.
- **Batching** — one multi-row insert instead of 1,000 single inserts; one **IN** lookup instead of N round-trips. Latency, not the database, is often the real cost.
- **Caching** — the fastest query is the one you never send.

A method for tuning a slow query

1. Find it — slow query log, or the engine's statement statistics view. Optimise by *total time* (calls × mean), not just the slowest single call.
2. **EXPLAIN ANALYZE** it. Find the operator where actual time or actual rows explodes.
3. Compare **estimated vs actual rows**. A big gap means bad statistics or a non-sargable/correlated predicate — fix that first; the plan often fixes itself.
4. Make the predicate sargable, add or reshape an index (consider covering it), or restructure the query.

5. Re-measure on realistic data volume, and check you haven't slowed down writes with a new index.

8 Execution plans

The plan is the optimizer showing its work: a tree of operators, each pulling rows from its children. Reading one is the difference between guessing and knowing.

EXPLAIN vs EXPLAIN ANALYZE

- `EXPLAIN` — shows the plan with **estimates**. Does not run the query. Safe on anything.
- `EXPLAIN ANALYZE` — **actually runs** the query and shows estimates alongside real row counts and timings. Far more useful. Careful: it really executes, so wrap a data-modifying statement in a transaction you roll back.

How to read the tree

Plans print as an indented tree. **Read it inside-out and bottom-up:** the most indented nodes run first and feed their parents. The top line is the last thing that happens.

Operators you'll actually see

NODE	WHAT IT DOES	REACTION
Seq / Table scan	Reads every page of the table	Fine on small tables or when returning most rows; a problem on a large table with a selective filter
Index scan / seek	Descends the index, then fetches matching rows from the table	Good, unless it fetches a huge number of rows one at a time
Index-only scan	Answers entirely from the index — never touches the table	Best case. This is a covering index working.
Bitmap index scan + heap scan	Collects matching row locations into a bitmap, sorts them, then reads pages in physical order	The middle ground for medium-selectivity filters; turns random I/O into sequential
Nested loop	Per outer row, look up the inner side	Great with a small outer side and an index; disastrous when loops is huge

NODE	WHAT IT DOES	REACTION
Hash join (+ Hash)	Builds a hash table on one input, probes with the other	Right for big joins; watch for spilling to disk
Merge join	Walks two sorted inputs together	Fine; check whether the sorts feeding it are expensive
Sort	Orders rows for ORDER BY, merge join, or dedup	Check whether it spilled to disk; an index can sometimes remove it entirely
Aggregate / HashAggregate	GROUP BY and aggregate functions	Normal; large hash aggregates use lots of memory
Limit	Stops after N rows	Good — it lets children stop early
Materialize / temp table	Buffers an intermediate result	Often a sign a subquery couldn't be flattened

The numbers on each line

- **Startup cost** — work before the first row appears (a sort must finish entirely; a scan can emit immediately).
- **Total cost** — work to return all rows. Arbitrary units, useful only for comparing plans on the same system.
- **rows** — the estimate. **actual rows** — what really came out.
- **loops** — how many times this node ran. In most formats the per-node times and rows are *averages per loop*, so multiply by `loops` for the true total. Missing this is the most common misreading of a plan.
- **width** — estimated average row size in bytes.
- **buffers / reads** — pages touched, and how many came from disk rather than cache. The most honest measure of work done.

Worked example

```
-- The query
SELECT c.name, o.id, o.total
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE c.city = 'Pune' AND o.created_at >= '2026-01-01';

-- Plan (slow: 4.2 s)
Hash Join (cost=..) (actual time=812..4193 rows=48210 loops=1)
  Hash Cond: (o.customer_id = c.id)
    -> Seq Scan on orders o (actual rows=9840000 loops=1)
        Filter: (created_at >= '2026-01-01')
        Rows Removed by Filter: 140160000
    -> Hash (actual rows=1210 loops=1)
        -> Seq Scan on customers c (actual rows=1210 loops=1)
            Filter: (city = 'Pune')
```

Diagnosis. The engine read 150 million order rows and threw away 140 million of them — there is no usable index on `orders.created_at`. The customers side is fine (1,210 rows is small). The hash join itself is a reasonable choice given the inputs.

```
-- The fix
CREATE INDEX idx_orders_created_customer
ON orders (created_at, customer_id) INCLUDE (id, total);

-- Plan (fast: 95 ms)
Hash Join (actual time=6..95 rows=48210 loops=1)
  -> Index Only Scan using idx_orders_created_customer on orders o
        (actual rows=9840000 loops=1) Heap Fetches: 0
        Index Cond: (created_at >= '2026-01-01')
  -> Hash (actual rows=1210 loops=1)
        -> Index Scan using idx_customers_city on customers c
```

Red-flag checklist

- **Estimated rows off from actual by 100× or more.** The most valuable signal in any plan — every downstream choice was made on a false premise. Refresh statistics, make predicates sargable, add multi-column statistics for correlated columns.
- **Nested loop with a large `loops` count.** Usually the symptom of the above.
- **"Rows Removed by Filter" far larger than rows returned.** You're reading data to throw it away — index the filter column.
- **Sort or hash spilling to disk** ("external merge", "batches > 1"). Increase working memory, reduce the row width, or avoid the sort with an index.

- **Index scan followed by many heap fetches.** Make the index cover the query.
- **A sequential scan on a large table with a selective `WHERE`.** Missing or unusable index.
- **Time concentrated in one node.** Optimise that node; ignore the rest.

SAY THIS

"I start from actual vs estimated rows, then find the node holding most of the time, then decide whether the fix is statistics, an index, or a query rewrite — and I re-run `EXPLAIN ANALYZE` to confirm rather than assuming."

9 Connection pooling

A connection pool is a set of already-open database connections that your application borrows and returns instead of opening a new one per request.

Why a connection is expensive

- **Setup latency:** TCP handshake, then TLS negotiation, then authentication, then session setup. Tens of milliseconds — often more than the query itself.
- **Server-side memory:** each connection gets a backend process or thread plus per-session work memory and caches. Hundreds of idle connections can cost gigabytes.
- **Context switching:** far more concurrent connections than CPU cores means the OS thrashes between them. Throughput *falls* past a certain point even as connections rise — the classic hockey-stick-in-reverse latency curve.
- **Hard limits:** every server has a max-connections cap; exceeding it means outright connection errors for everyone.

The pool lifecycle

```

acquire() → borrow an idle connection
            (or open a new one if under max,
             or wait up to the acquire timeout, then error)
    ↓
use it    → run statements / a transaction
    ↓
release() → return it to the pool -- MUST happen, even on exception
            (pool resets session state: rollback any open txn)

```

Always release in a `finally` block or a language construct that guarantees it (`try-with-resources`, `with`, `defer`). A leaked connection is never returned, and enough leaks starve the pool.

The settings that matter

SETTING	WHAT IT CONTROLS	SENSIBLE STARTING POINT
max pool size	Ceiling on concurrent connections from this instance	Small. See the sizing note below.
min idle	Connections kept warm even when quiet	Equal to max for steady load, so bursts don't pay setup cost
acquire / connection timeout	How long a caller waits for a free connection before failing	1-5 s. Fail fast — queueing forever turns into a cascading outage
idle timeout	Closes connections unused for this long	Minutes; only useful if min < max
max lifetime	Retires a connection after this age regardless of health	Shorter than any network/database idle-kill timeout, so <i>you</i> close it, not a firewall
validation / keepalive	Tests a connection before handing it out	Use the driver's lightweight ping, not a full <code>SELECT 1</code> per checkout, if available
leak detection threshold	Logs a stack trace when a connection is held too long	Enable in non-production; it finds leaks instantly
statement timeout	Kills a query that runs too long	Set it. One runaway query otherwise holds a pooled connection forever

Sizing: smaller than you think

TWO FORMULAS

Capacity-based (the well-known rule of thumb):

$\text{connections} \approx (\text{cpu_cores} \times 2) + \text{effective_spindle_count}$

For an 8-core server on SSD that is roughly 16–20 *total* connections doing real work. A pool of 200 is almost always wrong.

Demand-based (Little's Law):

$\text{pool_size} \approx \text{throughput (req/s)} \times \text{average_holding_time (s)}$

500 req/s each holding a connection for 10 ms needs about 5 connections. If that surprises you, the intuition to keep is: a small pool with a queue beats a large pool that thrashes — the work gets done either way, but a small pool keeps latency predictable.

THE MULTIPLICATION TRAP

Pool size is *per application instance*. 20 pods × a pool of 50 = 1,000 connections at the database, which may be above its limit. Always compute $\text{instances} \times \text{max_pool_size}$ (plus replicas, plus migration jobs, plus your admin session) against the server's cap. Autoscaling makes this worse silently.

Client-side pool vs external pooler

In-process pool (library)

Lives inside your app. Zero extra hop, simple. But each instance has its own pool with no global view, so the total is whatever your fleet size happens to be.

External pooler (proxy)

A middleware process in front of the database. Gives a single global limit across all clients, multiplexes many client connections onto few server ones, and survives app restarts. Costs an extra network hop and a component to operate.

POOLING MODES ON AN EXTERNAL POOLER

- **Session pooling** — a server connection is dedicated to a client until it disconnects. Safest; least multiplexing benefit.
- **Transaction pooling** — the server connection is assigned only for the duration of a transaction, then reused. Huge multiplexing win, and the usual choice.
- **Statement pooling** — reassigned per statement. Most aggressive; multi-statement transactions are not possible.

TRANSACTION POOLING BREAKS SESSION STATE

Because consecutive statements may land on different server connections, anything that lives in the *session* stops working: session-level `SET`, temporary tables, advisory/session locks, `LISTEN/NOTIFY`, cursors held outside a transaction, and server-side prepared statements (unless the pooler explicitly supports them). If your ORM uses prepared statements by default, you must either disable them or enable the pooler's support — a very common production surprise.

Pool exhaustion: symptoms and causes

Symptom: "timeout waiting for connection from pool", latency climbing while database CPU is *low*. That combination — slow app, idle database — is the signature of a pool problem rather than a query problem.

- **Leaks** — a code path that doesn't return the connection.
- **Long transactions** — a transaction left open across an HTTP call to a third party holds its connection (and its locks, and MVCC cleanup) for the whole round trip. Never do I/O inside a transaction.
- **N+1 queries** — multiplying the number of statements per request, so holding time balloons.
- **A missing index** turning a 5 ms query into 2 s, so each connection is held 400× longer. Pool exhaustion is often a query problem wearing a disguise.
- **Pool smaller than concurrency**, or an acquire timeout so long that callers pile up instead of failing fast.

Serverless and short-lived compute

Functions that scale to hundreds of instances, each with its own pool, will exhaust a database's connection limit almost immediately — and each cold start pays full connection setup. The standard answers: put an external pooler (or the provider's managed data proxy) between them, keep per-instance pools at 1-2, and reuse the client across invocations rather than constructing it per request.

Operational hygiene

- Export pool metrics: active, idle, waiting, and acquire wait time. **Waiting count** is the leading indicator of trouble.
- Set both an acquire timeout and a statement timeout, so a single bad query can't consume the pool.
- Retry only *idempotent* operations, with exponential backoff and jitter; a circuit breaker stops a struggling database being hammered.

- Separate pools for separate workloads — keep batch/reporting traffic out of the pool serving user requests, and point read-only traffic at replicas.
- Set `max lifetime` below any idle-connection timeout in the network path, or you'll hand out sockets that were silently killed.

10 Interview questions — Part II

Same drill. These lean towards "here is a slow query, what do you do", which is how the second round usually goes.

22 QUESTIONS · JOINS, OPTIMIZER, PLANS, POOLING

01 Explain the join types.

INNER returns only matching pairs. LEFT OUTER returns all left rows with NULLs where the right has no match (RIGHT is its mirror). FULL OUTER returns unmatched rows from both sides. CROSS returns the Cartesian product. A SELF join is a table joined to itself with aliases, for hierarchies. Beyond those, two shapes matter in plans: a *semi-join* (left rows that have a match, no duplication — from `EXISTS` / `IN`) and an *anti-join* (left rows with no match — from `NOT EXISTS`).

Testing: baseline. Naming semi/anti-join marks you out.

02 Difference between putting a condition in ON versus WHERE.

For an INNER join, none — the result is identical. For an OUTER join it's decisive: `ON` is applied while matching, `WHERE` after the join is formed. A `WHERE` condition on the optional table rejects the NULL-extended rows and silently converts your LEFT join into an INNER join. So conditions on the optional side go in `ON`; conditions on the preserved side go in `WHERE`.

Testing: the single most common SQL bug in production code.

03 Why does NOT IN sometimes return nothing?

Because of three-valued logic. If the subquery returns any NULL, then `x NOT IN (...)` evaluates to unknown for every x — it can never be true, so zero rows come back. `NOT EXISTS` is NULL-safe and typically optimises into an anti-join, so it is both correct and usually faster. A LEFT JOIN with `IS NULL` is the third equivalent form.

Testing: NULL semantics — a favourite because it separates memorisers from users.

04 Describe nested loop, hash and merge joins, and when each is picked.

Nested loop: for each outer row, find matches in the inner. $O(N \times M)$, but with an index on the inner join column it's $O(N \log M)$ — the OLTP default when the outer side is small. **Hash join:** build a hash table on the smaller input, probe with the larger. $O(N+M)$, equality conditions only, needs memory and spills to disk if the build side is too big — the choice for large unsorted inputs. **Merge join:** sort both sides on the key and walk them together. Good when inputs arrive sorted, for very large joins, and for non-equality conditions; its output stays sorted.

Short version: few rows → index nested loop; many rows → hash; already sorted or inequality → merge.

Testing: whether you can read a plan and judge whether the algorithm chosen is sane.

05 A join produces more rows than expected and the SUMs are wrong. Why?

Fan-out. Joining a parent to two different one-to-many children multiplies their rows into a cross-product, so every aggregate is inflated by the other child's row count. The fix is to aggregate each child separately — in a CTE or derived table — and join the single-row results to the parent. Slapping `DISTINCT` on top hides the symptom and usually still gets the sums wrong.

Testing: a real debugging scenario. "DISTINCT is a smell" is the sentence they want.

06 How does a query get from text to results?

Parse to a tree; bind names and types and check permissions; rewrite into an equivalent but cheaper logical form (expand views, flatten subqueries, push predicates down, prune columns); optimize — enumerate access methods, join algorithms and join orders, cost each using statistics, pick the cheapest; then execute the physical plan. Repeated queries can skip most of that via a plan cache, which is part of why prepared statements help.

Testing: that you know optimization is a distinct, cost-based phase.

07 What is a cost-based optimizer and what can mislead it?

It generates candidate plans and estimates each one's cost from statistics — row counts, distinct values, null fractions, histograms — then executes the cheapest. It is misled by: stale statistics after bulk loads or deletes; correlated columns, because it usually assumes independence and multiplies selectivities into an underestimate; non-sargable predicates it can't reason about; opaque function calls; parameter-sniffing effects where a cached plan built for one parameter value is terrible for another; and very wide joins where it stops searching exhaustively.

Testing: depth. Correlated columns and parameter sniffing are strong signals.

08 What does "sargable" mean?

Search-ARGument-able: the predicate is written so an index seek can be used. Keep the indexed column bare on one side. `WHERE created_at >= '2026-01-01'` is sargable; `WHERE YEAR(created_at) = 2026`, `WHERE price * 1.18 > 1000` and `WHERE CAST(id AS text) = '42'` are not, because the index stores the column, not the expression. Fix by rewriting into a range, moving the arithmetic to the constant side, matching parameter types, or creating an expression index.

Testing: vocabulary plus the rewrite instinct.

09 How would you paginate a large result set efficiently?

Not with large `OFFSET`s — the engine still reads and discards every skipped row, so page 5,000 is thousands of times slower than page 1, and rows shift under the user as data changes. Use **keyset (seek) pagination**: remember the last row's sort key and fetch `WHERE (created_at, id) < (:last_created, :last_id) ORDER BY created_at DESC, id DESC LIMIT 20`, with an index on that exact tuple. Constant time per page and stable under concurrent inserts. The trade-off is no random jump to page N, which is fine for infinite scroll and APIs.

Testing: a practical scaling question. The tie-breaker column shows care.

10 EXISTS vs IN vs JOIN for "customers who have orders".

All three can produce the same rows, and modern optimizers usually converge on a semi-join for `EXISTS` and `IN`. Prefer `EXISTS`: it short-circuits at the first match and is NULL-safe, which matters enormously in the negated form. A plain `JOIN` is different in one respect — it *duplicates* a customer once per order, so you'd need `DISTINCT`, which adds a sort or hash. Use a join when you actually want columns from the other table.

Testing: that you know the duplication difference, not just "they're the same".

11 Difference between WHERE and HAVING.

`WHERE` filters individual rows before grouping and can use indexes; `HAVING` filters groups after aggregation and can reference aggregate functions. Anything that doesn't need an aggregate should be in `WHERE` — filtering later means grouping rows you were going to discard.

Testing: basic, but they check you mention the performance implication.

12 When is a window function better than a self-join or correlated subquery?

Whenever you need a value computed over a group but still want row-level output: running totals, ranking, "top N per group", comparing a row to its group's average, or previous/next values with `LAG` / `LEAD`. A correlated subquery re-executes per output row — N+1 inside SQL — whereas a window function typically needs one sort or hash over the data. It's usually both faster and clearer.

Testing: whether your SQL is modern.

13 EXPLAIN vs EXPLAIN ANALYZE.

`EXPLAIN` shows the chosen plan with estimated costs and row counts without running the query. `EXPLAIN ANALYZE` executes it and reports actual times, actual rows and loop counts alongside the estimates — which is what you need, because the gap between estimated and actual is the diagnosis. Since it really runs, wrap a data-modifying statement in a transaction and roll back.

Testing: that you know `ANALYZE` executes. People get caught by this.

14 How do you read an execution plan?

Bottom-up and inside-out: the most indented nodes run first and feed their parents. For each node check actual rows against the estimate, the time spent, and the `loops` count — per-node figures are typically averages per loop, so multiply. Then find where the row count or the time explodes; that node is the problem, and everything else is noise. If buffer/read counts are available, they're the most honest measure of work.

Testing: reading direction and the loops multiplication — both commonly got wrong.

15 What are your red flags in a plan?

Estimated rows far off actual (100×+) — the root cause of most bad plans. A nested loop with a huge loop count. "Rows removed by filter" dwarfing rows returned. Sorts or hashes spilling to disk. An index scan followed by masses of heap fetches, meaning the index doesn't cover the query. A sequential scan on a big table with a selective predicate. And one node holding almost all of the elapsed time.

Testing: a checklist rather than one memorised item.

16 A query got slow overnight with no code change. What happened?

Most likely the plan changed. Candidates: data volume grew past a tipping point so the optimizer switched strategies; statistics went stale after a bulk load or purge; a cached plan was rebuilt with an unrepresentative parameter (parameter sniffing); an index was dropped, disabled or became bloated; the buffer cache is now too small for a grown working set, turning cached reads into disk reads; or lock contention from a new background job. I'd compare the current plan to the expected one, refresh statistics, and check locks and I/O metrics.

Testing: systematic diagnosis under ambiguity — more valuable than any single fact here.

17 What is partitioning and when does it help?

Splitting one logical table into physical pieces by range (usually a date), list, or hash. Two real benefits: **partition pruning**, where a query with a predicate on the partition key touches only relevant partitions; and **lifecycle management** — dropping last year's data is an instant partition drop rather than a giant DELETE with its vacuum aftermath. It doesn't help if queries never filter on the partition key, and it can hurt: cross-partition queries touch every piece, and unique constraints must include the partition key.

Testing: that you know partitioning isn't a general speed-up.

18 Why use a connection pool?

Because opening a connection costs a TCP handshake, TLS negotiation, authentication and session setup — often more than the query — and each open connection consumes server memory and a process or thread. Pooling amortises setup across requests and, just as importantly, *caps* concurrency, so the database isn't thrashing between hundreds of connections on a handful of cores.

Testing: that you name both benefits — latency and the concurrency ceiling.

19 How do you size a pool?

Smaller than instinct suggests. Capacity rule of thumb: $(\text{cores} \times 2) + \text{effective spindles}$, so ~16–20 total for an 8-core SSD server. Demand-based via Little's Law: $\text{pool} = \text{throughput} \times \text{holding time}$, so $500 \text{ req/s} \times 10 \text{ ms} \approx 5$ connections. Then multiply by instance count and check the total against the server's max-connections, remembering replicas, migrations and admin sessions.

The reasoning: beyond the point where the database can execute work in parallel, extra connections only add context switching and memory pressure — queueing in the pool gives more predictable latency than queueing inside the database.

Testing: a formula, and the fleet-multiplication check that most candidates forget.

20 Requests are timing out waiting for a connection but database CPU is low. Diagnose.

That combination points at holding time, not database capacity. Check for connection leaks (paths that don't release in a `finally`); transactions left open across external HTTP calls; N+1 query patterns inflating statements per request; and a slow query — often a newly missing or unused index — that turned 5 ms of holding time into 2 s. Then check whether the pool is simply smaller than real concurrency and whether the acquire timeout is so long that callers queue instead of failing fast. Enable leak detection, add a statement timeout, and look at the pool's *waiting* metric.

Testing: whether you reason from the evidence (low CPU) rather than reflexively raising the pool size.

21 What breaks under transaction-mode pooling?

Anything that assumes a stable session, because consecutive transactions may run on different server connections: session-level `SET`, temporary tables, session/advisory locks, `LISTEN/NOTIFY`, cursors held outside a transaction, and server-side prepared statements unless the pooler explicitly supports them. The usual fixes are to enable the pooler's prepared-statement support or disable them in the driver, set session parameters per transaction, and use session-mode pooling for the few clients that genuinely need session state.

Testing: production experience. This is the question that separates read-about-it from ran-it.

22 Serverless functions are exhausting the database's connections. What do you do?

Each instance carries its own pool and they scale horizontally without coordination, so the fleet total blows past the server limit — and every cold start pays full connection setup. Put an external pooler or managed data proxy in front so there's one global cap and server connections are multiplexed; keep per-instance pools at 1-2; create the client outside the handler so it's reused across warm invocations; add a short acquire timeout with retry and backoff; and, where the workload allows, use an HTTP/data-API path that doesn't hold a session at all.

Testing: modern deployment awareness — increasingly a standard question.

★ Night-before sheet

If you read nothing else, read this. Highest-yield facts, in the order you're most likely to need them.

NORMALIZATION

- 1NF atomic · 2NF no partial · 3NF no transitive · BCNF every determinant is a super key.
- "The key, the whole key, and nothing but the key."
- 3NF does *not* imply BCNF — overlapping candidate keys are the counter-example.
- Anomalies: insert, update, delete. Say all three.
- Denormalize with a measured reason and a named sync mechanism.

INDEXING

- B+ tree: high fanout, 3-4 levels, data in sorted linked leaves → ranges and ORDER BY.
- Hash: $O(1)$ equality only. No ranges, no ordering, no prefixes.
- Leftmost prefix; equality columns first, range column last.
- Covering index → index-only scan → no table reads.
- Not used because: non-sargable, leading wildcard, implicit cast, low selectivity, stale stats.
- Every index slows every write.

TRANSACTIONS

- A = undo log · C = constraints + invariants · I = isolation level · D = WAL + fsync.
- WAL: log before data page; commit after flush; checkpoint later; redo + undo on recovery.
- Strict 2PL holds exclusive locks until commit.
- Deadlock → wait-for cycle → victim → **retry**. Prevent with consistent lock ordering.

ISOLATION

- RU: dirty · RC: non-repeatable + phantom · RR: phantom (per standard) · SER: none.
- Non-repeatable = same row changed. Phantom = same range, different rows.
- Snapshot isolation still allows **write skew** → needs SERIALIZABLE or an explicit lock.
- MVCC: readers don't block writers; cost is version cleanup.
- Lost update: atomic statement, FOR UPDATE, or version compare-and-set.

JOINS

- Outer join + condition in WHERE = accidental inner join. Put it in ON.
- `NOT IN` + NULL = zero rows. Use `NOT EXISTS`.
- Two one-to-many children = fan-out = inflated SUMs.
- Few rows → index nested loop · many → hash · sorted/inequality → merge.

PLANS & TUNING

- Read bottom-up, inside-out. Multiply per-node figures by `loops`.
- First check: estimated vs actual rows. A 100× gap explains almost everything.
- Keyset pagination beats large OFFSET.
- EXPLAIN ANALYZE actually runs the query.

POOLING

- Size $\approx$ (cores $\times$ 2) + spindles, or throughput $\times$ holding time. Small.
- Multiply by instance count and compare to max_connections.
- Slow app + idle database CPU = pool problem, not query capacity.
- Transaction-mode pooling breaks prepared statements, temp tables, session SET.
- Always: release in `finally`, acquire timeout, statement timeout.

THREE SENTENCES THAT ALWAYS LAND

- "Indexes make reads faster and writes slower — that's the trade."
- "I'd check estimated versus actual rows before changing anything."
- "Deadlocks and serialization failures are expected; the application retries them."

Two days, nine topics, forty-six questions. Answer each question out loud before opening it — recognising an answer and being able to say it are different skills, and only one of them is tested in an interview.